Embedding và dữ liệu chuỗi

Padding để lưu, mask để hiểu đúng

Chuỗi ngắn, chuỗi dài

Tên bài có 2 token hoặc 6 token.

Batch cần một bảng chữ nhật.

Thêm PAD không thêm nội dung.

Câu hỏi: thêm PAD có đổi biểu diễn của câu không?

01

ID và lookup

ID là địa chỉ

Token “đồ_thị” có ID 2; “dp” có ID 3.

3 không lớn hơn 2 về ý nghĩa.

Embedding chọn hàng trong bảng `(V,d)`.

V là số token trong vocabulary; d là số thành phần của mỗi vector.

HỎI NHANH

Một bạn đưa thẳng cột ID, với đồ_thị = 2, dp = 3, toán = 4, vào `nn.Linear(1, 8)` như một feature số. Câu nào đúng?

A Linear tự học một vector riêng cho từng ID, giống Embedding

B Đầu ra là hàm tuyến tính của ID, nên đánh số lại là đổi model

C Chuẩn hóa ID bằng mean và std trước thì thành lookup đúng nghĩa

D Không sao, nếu ID được gán theo thứ tự tần suất xuất hiện

HỎI NHANH

Vocabulary có 50 000 token, embedding rộng 300, batch token ID có shape `(32, 128)`. Nếu tạo one-hot float32 rồi nhân với bảng thay cho lookup, riêng tensor one-hot chiếm khoảng bao nhiêu?

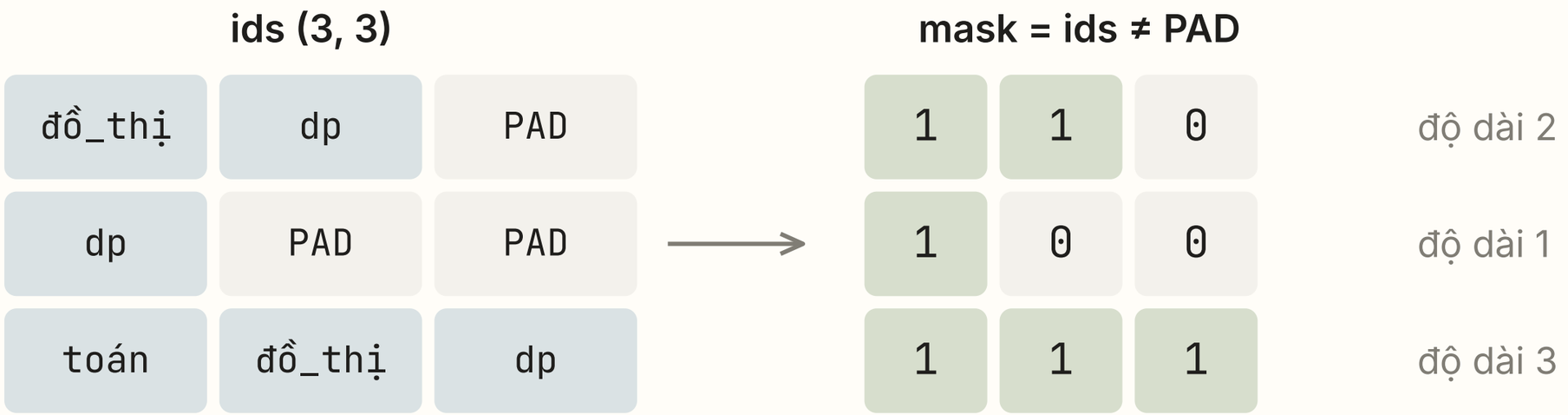
A Khoảng 4.9 MB, bằng đầu ra lookup `(32, 128, 300)`

B Khoảng 60 MB, bằng bảng embedding `(50000, 300)`

C Khoảng 819 MB, so với 4.9 MB của đầu ra lookup

D Khoảng 16 KB, vì one-hot chỉ lưu một số 1 cho mỗi token

Bảng chữ nhật và valid mask



bảng rộng bằng câu dài nhất trong batch; ô xám là PAD

Ba câu của bài sau pad_sequence thành bảng (3, 3); mask đánh dấu 1 ở token thật, nên tổng mỗi hàng chính là độ dài câu.

Batch trong notebook

KẾT QUẢ

```
[[2, 3, 0], [3, 0, 0], [4, 2, 3]]
[[1, 1, 0], [1, 0, 0], [1, 1, 1]]
lengths: [2, 1, 3]
```

`PAD = 0` · `UNK = 1`.

UNK là token thật chưa biết, PAD chỉ điền bảng.

Bảng dài bằng câu dài nhất **trong batch**.

HỎI NHANH

Một batch gồm ba câu dài 5, 12 và 7 token. Câu dài nhất của cả dataset có 200 token. `pad_sequence(..., batch_first=True)` cho ra tensor shape gì, và có bao nhiêu ô PAD?

A `(3, 12)`, 12 ô PAD

B `(12, 3)`, 12 ô PAD

C `(3, 12)`, không lưu ô PAD nào

D `(3, 200)`, 576 ô PAD

Lookup giữ các vị trí

ID `(B,L)` → vector `(B,L,d)`.

B: số chuỗi trong batch; L: độ dài sau padding; d: embedding width.

Ví dụ batch `(3,3)`, bảng `(5,2)`.

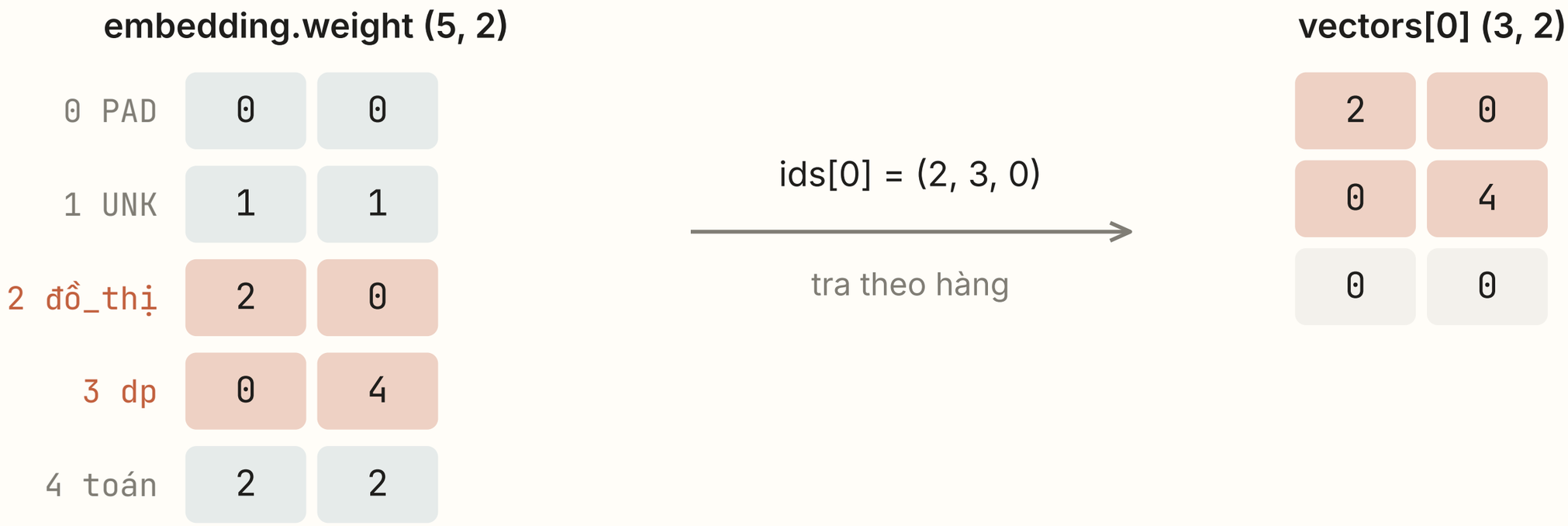
Câu đầu sau lookup:

KẾT QUẢ

```
[[2.0, 0.0], [0.0, 4.0], [0.0, 0.0]]
```

Vector đặt tay để dễ đọc; model thật học chúng.

Tra theo hàng



ID là địa chỉ hàng, không phải độ lớn

Bảng embedding đặt tay của bài: câu đầu (2, 3, PAD) tra ra ba hàng của bảng, nên vectors có thêm chiều cuối d = 2.

HỎI NHANH

Input ID có shape `(8, 12)`, embedding rộng 16. Đầu ra có shape gì?

A `(8, 16)`

B `(12, 16)`

C `(8, 16, 12)`

D `(8, 12, 16)`

HỎI NHANH

Tạo `nn.Embedding(5, 2, padding_idx=0)`, rồi nạp một checkpoint có hàng 0 bằng `[0.3, -0.1]`. Lookup một ô PAD trả về gì, và masked mean của bài còn đúng không?

A `[0, 0]`, masked mean vẫn đúng

B `[0, 0]`, masked mean sai theo

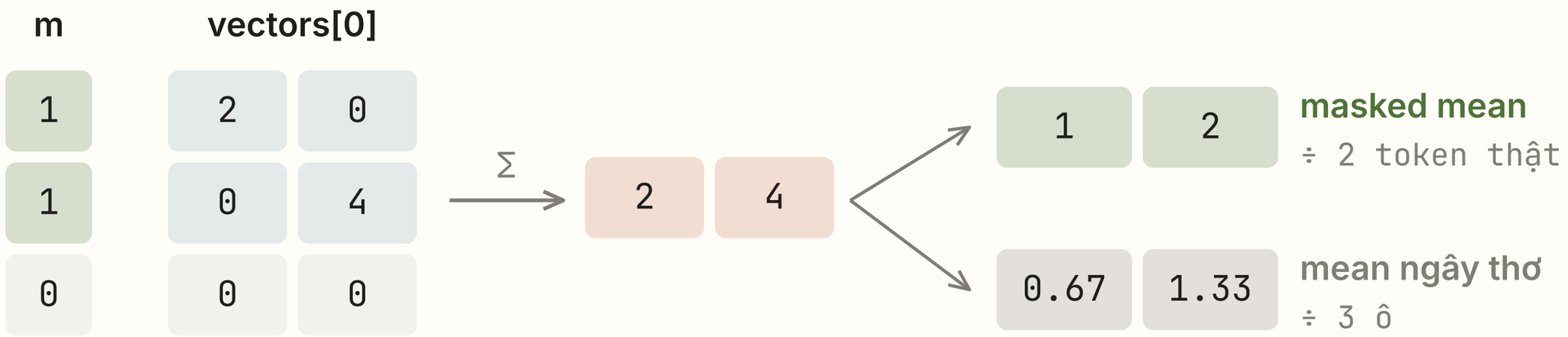
C `[0.3, -0.1]`, masked mean vẫn đúng

D `[0.3, -0.1]`, masked mean sai theo

02

Gộp đúng token thật

Tổng đúng, mẫu số cũng phải đúng



Masked mean của câu đầu: cộng vector theo mask được (2, 4), chia cho 2 token thật ra (1, 2); chia cho cả 3 ô thì ra (0.67, 1.33).

Masked mean

$$\mathbf{s}_i = \frac{\sum_t m_{i,t}\mathbf{e}_{i,t}}{\sum_t m_{i,t}}.$$

$m_{i,t} = 1$ ở token thật, bằng 0 ở PAD; $\mathbf{e}_{i,t}$ là vector token. Mẫu số là số token thật và phải khác 0.

Mask thêm chiều cuối trước nhân với vectors.

Gộp chiều độ dài, giữ batch và đặc trưng.

Ví dụ tính tay

$([2, 0] + [0, 4])/2 = [\mathbf{1}, \mathbf{2}].$

Mean cả PAD: $[2/3,\ 4/3]$.

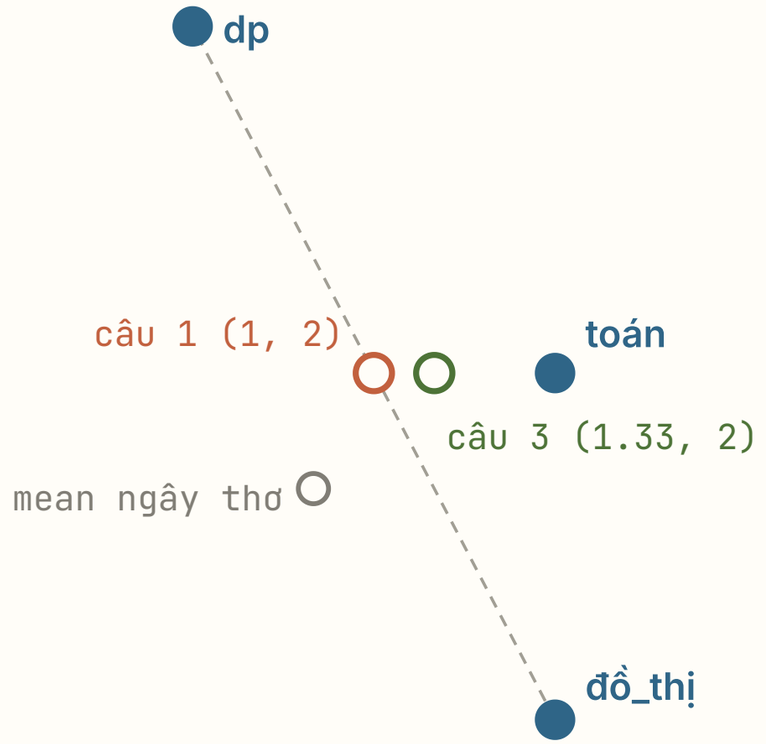
Trong output notebook:

```
masked[0] = [1.0, 2.0]
```

```
naive first = [0.6667, 1.3333]
```

PAD bằng 0 vẫn làm sai mẫu số nếu không mask.

Ba câu trong mặt phẳng



Vector token đặt tay trong mặt phẳng: câu 1 là trung điểm của đồ_thị và dp; mean ngây thơ chia cho 3 ô (tính cả PAD) kéo điểm về phía gốc.

HỎI NHANH

Vector của PAD bằng 0. Hai token thật có tổng vector là $(4, 6)$. Câu được pad thêm hai PAD, và code lấy mean trên cả bốn vị trí, ra $(1, 1.5)$. Sai ở đâu?

A Ở tử số, vì hai PAD đã cộng thêm vào tổng

B Ở mẫu số, vì chia cho 4 vị trí thay vì 2 token thật

C Ở mask, vì mask chỉ cần khi PAD mang ID âm

D Không sai, vì PAD bằng 0 nên mean nào cũng như nhau

HỎI NHANH

Trong `masked_mean`, một bạn bỏ `keepdim=True`, nên `counts` có shape `(B,)`. Chạy với hai câu đầu của bài, `[2, 3]` và `[3]`, tức $B = d = 2$, code không báo lỗi. Kết quả là gì?

```
counts = valid_mask.sum(dim=1)          # shape (2,)
return weighted.sum(dim=1) / counts     # (2,2) / (2,)
```

A `[[1, 2], [0, 4]]`, như bản đúng

B `[[1, 2], [0, 2]]`

C Báo lỗi broadcast, vì hai tensor khác số chiều

D `[[1, 4], [0, 4]]`

Hai phép kiểm

KẾT QUẢ

```
adding PAD changes nothing: True
empty sequence rejected: True
```

Thêm PAD không đổi thông tin.

Câu rỗng cần quy tắc rõ, tránh chia 0.

HỎI NHANH

Thay `raise ValueError` bằng `counts.clamp_min(1)`, rồi chạy trên một câu toàn PAD. Chuyện gì xảy ra?

A Lỗi chia cho 0, chương trình dừng ngay ở câu ấy

B Trả về `nan`, loss thành `nan` nên lỗi lộ ra ngay

C Trả về vector 0, rồi trôi vào classifier như một câu thật

D Trả về embedding của UNK, vì câu rỗng là token lạ

03

Học vector và thứ tự

Gradient theo số lần dùng

Loss kiểm là tổng vector lookup.

Token 2 xuất hiện 2 lần → gradient $[2, 2]$.

Token 3 xuất hiện 3 lần → gradient $[3, 3]$.

Hàng PAD nhận gradient 0 nhờ `padding_idx`.

Training thật: không `detach` vector `pooled` nếu muốn embedding học.

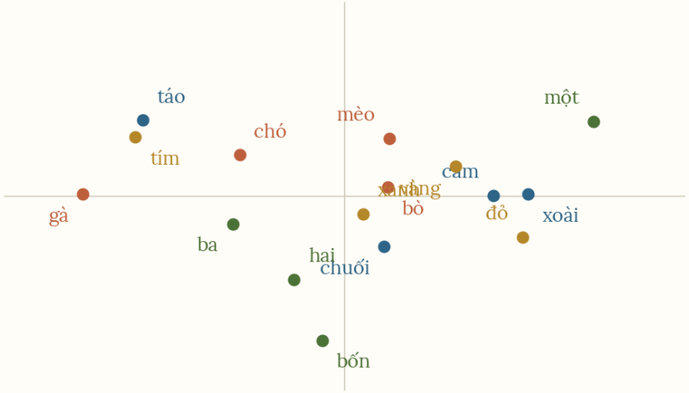
Ý nghĩa đến từ mục tiêu học

Word2vec: mỗi từ dự đoán các từ quanh nó, không cần nhãn.

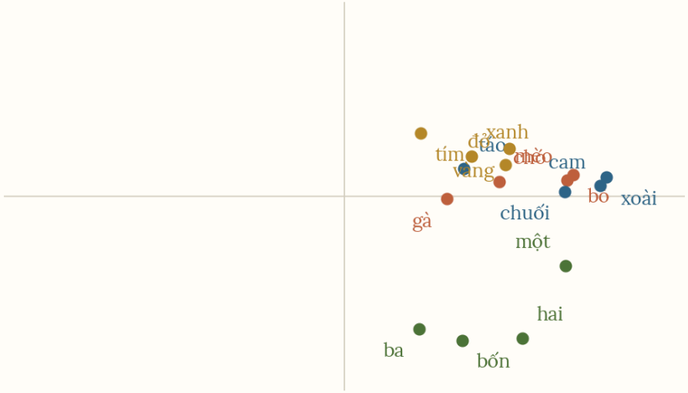
“BFS”, “DFS” cùng ngữ cảnh → hai vector gần nhau.

ID vẫn chỉ là địa chỉ.

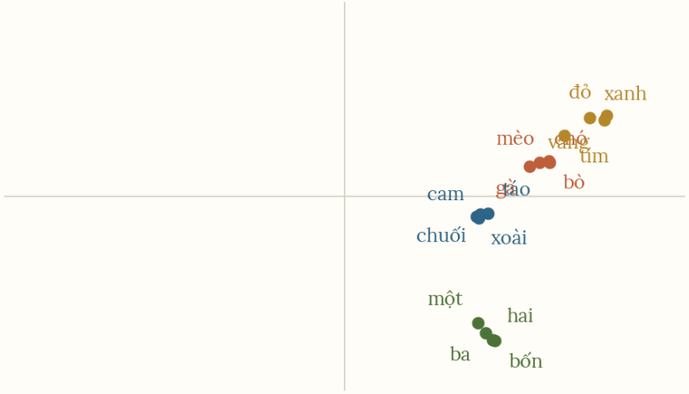
Skip-gram trên 64 câu ngắn



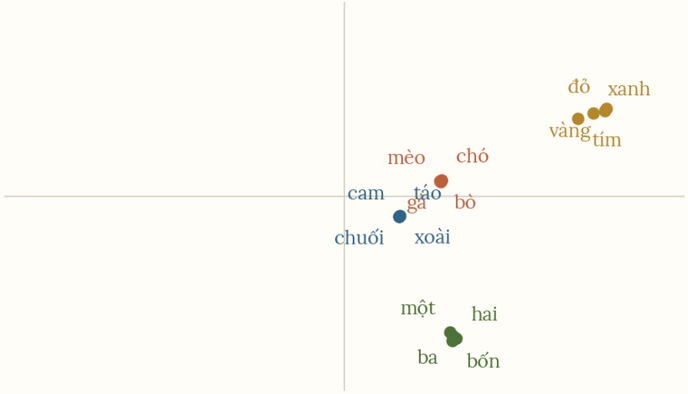
epoch 0: vector khởi tạo ngẫu nhiên



epoch 35



epoch 120



epoch 400: con vật, trái cây, con số, màu tách thành bốn nhóm

Embedding hai chiều học bằng skip-gram trên 64 câu ngắn: chỉ nhờ đoán từ đứng cạnh, các từ cùng loại dần đứng gần nhau.

HỎI NHANH

Với `padding_idx=0` và loss là `embedding(ids).sum()`, batch là `ids = [[4, 4, 0], [4, 2, 0]]`. Gradient của hàng 4 và của hàng 0 là gì?

A Hàng 4 nhận `[3, 3]`, hàng 0 nhận `[0, 0]`

B Hàng 4 nhận `[1, 1]`, hàng 0 nhận `[0, 0]`

C Hàng 4 nhận `[2, 2]`, hàng 0 nhận `[0, 0]`

D Hàng 4 nhận `[3, 3]`, hàng 0 nhận `[2, 2]`

Mean pooling mất thứ tự

“đồ_thị dp” và “dp đồ_thị” có cùng mean.

KẾT QUẢ

```
same after swapping: True
```

Token phải gặp vị trí **trước** pooling: RNN hoặc attention.

HỎI NHANH

Cộng một vector vị trí $\mathbf{p}_t$ vào embedding của token ở vị trí t, rồi lấy mean ngay. Hai câu “A thắng B” và “B thắng A” cho ra biểu diễn thế nào?

A Khác nhau, vì giờ mỗi token mang theo vị trí của nó

B Giống nhau, vì tổng các token và tổng các vị trí đều không đổi

C Khác nhau, vì A và B mang hai ID khác nhau

D Giống nhau, nhưng chỉ khi vector của A và B trùng nhau

04

RNN và chuỗi dài

RNN: vòng lặp có trạng thái

$$\mathbf{h}_t = \tanh\left(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}\right).$$

Cùng trọng số ở mọi bước; đổi thứ tự token là đổi kết quả.

$d = 2$, $H = 3$: 21 tham số (PyTorch giữ hai bias).

Đọc trạng thái đúng chỗ

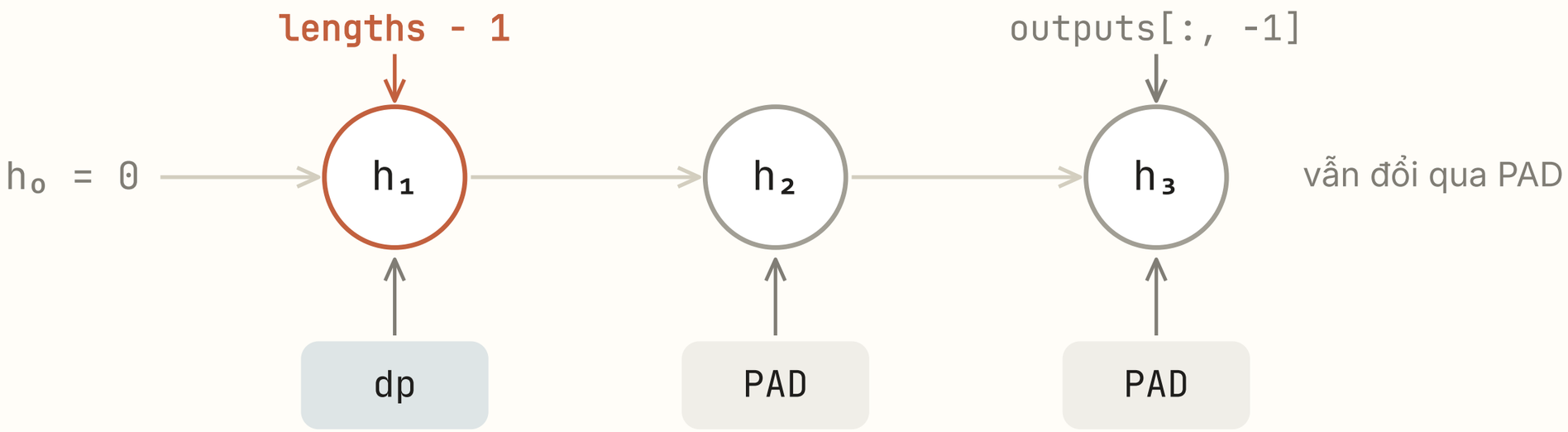
PAD vẫn cập nhật $\mathbf{h}$ qua W_{hh} và bias.

```
last_column = outputs[:, -1]
last_valid = outputs[torch.arange(len(ids)),
                     lengths - 1]
```

KẾT QUẢ

```
last column is last token: [False, False, True]
gather matches packed: True
RNN same after swapping: False
```

Câu “dp” được pad tới 3



Câu “dp” dài 1 trong bảng rộng 3: trạng thái cần đọc là h_1; cột cuối h_3 đã đi qua hai bước PAD và vẫn bị W_hh, bias làm đổi.

HỎI NHANH

Phân loại câu bằng RNN với `batch_first=True`, batch được pad bên phải. Đội A lấy `outputs[:, -1]`, đội B lấy `outputs[i, lengths[i]-1]`. Validation của đội A thấp hơn, và còn đổi khi xáo lại các batch. Vì sao?

A A đọc trạng thái đã chạy qua PAD, mà lượng PAD đổi theo batch

B A đọc trạng thái ở đầu chuỗi, khi chưa thấy các token sau

C B đọc nhầm PAD, vì `lengths` đếm cả token đặc biệt

D Hai cách như nhau, vì vector PAD bằng 0 ở mọi bước

Gradient tắt dần

KẾT QUẢ

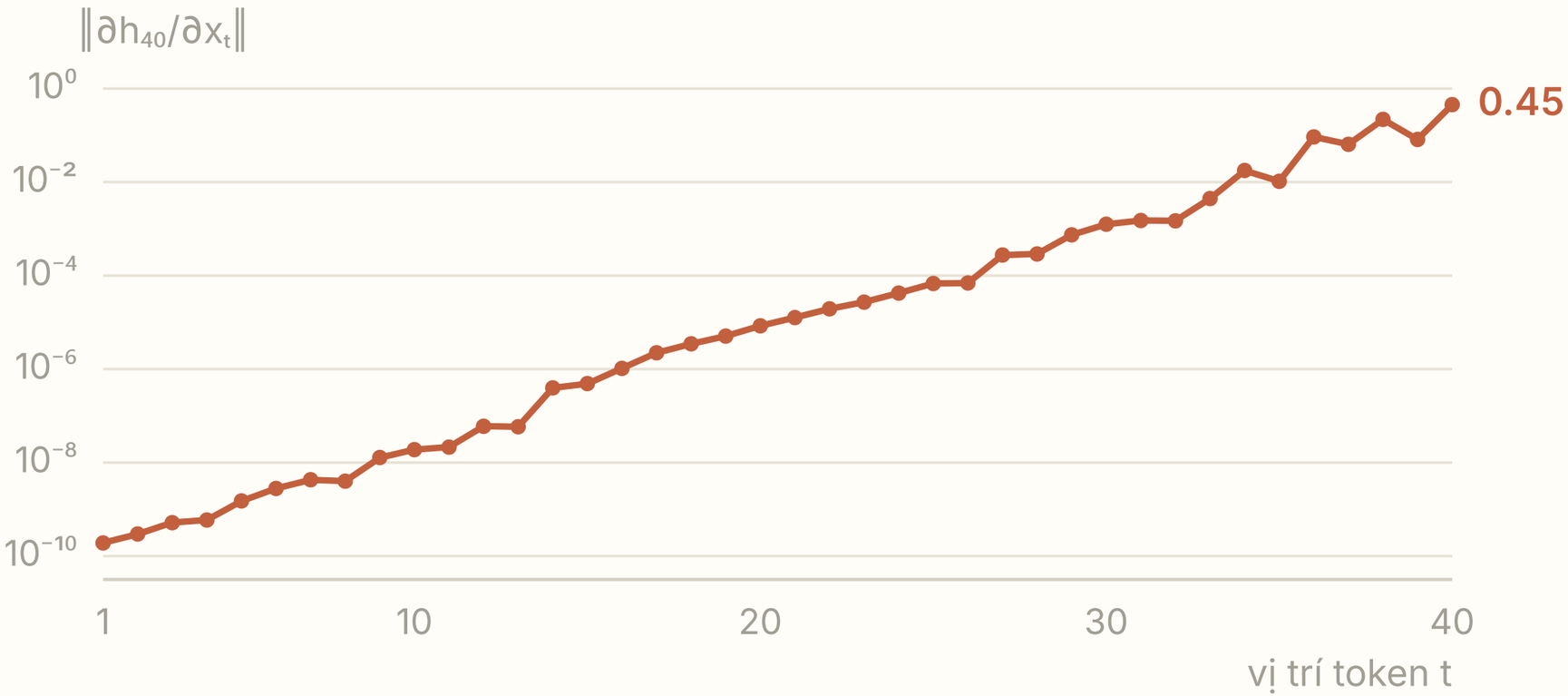
```
token 40: 4.5e-01
token 30: 1.3e-03
token 20: 8.4e-06
token 10: 1.9e-08
token  1: 1.9e-10
```

Mỗi bước nhân một Jacobian (W_{hh} và $\tanh'$).

Vô hướng: $\partial h_L / \partial x_t = w^{L-t}$.

$|w| < 1$ tắt; $|w| > 1$ bùng nổ → clipping.

Gradient theo vị trí token



RNN của bài trên chuỗi dài 40 (số từ code): độ lớn gradient của trạng thái cuối theo từng token giảm hai đến ba bậc mỗi mười bước, token đầu chỉ còn cỡ 10^{-10}.

HỎI NHANH

RNN một chiều số $h_t = w\,h_{t-1} + x_t$, chuỗi dài 30. Đạo hàm $\partial h_{30}/\partial x_1$ bằng bao nhiêu khi $w = 0.5$, và khi $w = 1.5$?

A $0.5 \cdot 29 = 14.5$ và $1.5 \cdot 29 = 43.5$

B Bằng 1 với mọi w, vì x_1 cộng thẳng vào trạng thái

C Khoảng $9.3 \cdot 10^{-10}$ và $1.9 \cdot 10^{5}$

D Khoảng $1.9 \cdot 10^{-9}$ và $1.3 \cdot 10^{5}$

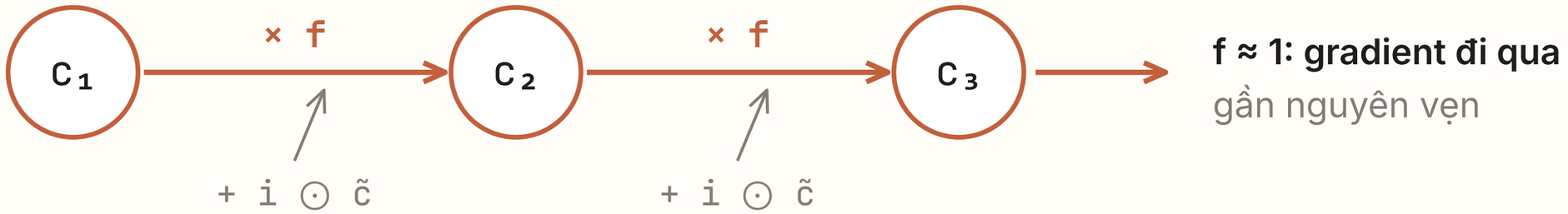
LSTM: đường cộng có cổng

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

Forget gate gần 1: gradient qua nhiều bước gần nguyên vẹn.

`nn.LSTM` gọi như `nn.RNN`; trạng thái cuối là cặp `(h_n, c_n)`. Vẫn tuần tự; attention (bài 3.9) nối thẳng mọi cặp.

Đường cell state



đường cell state chỉ nhân với cổng quên, không nhân W_hh và tanh′ mỗi bước

Cell state của LSTM: từ c_1 tới c_3 chỉ nhân với cổng quên f rồi cộng nội dung mới, nên khi f gần 1 gradient đi qua nhiều bước mà không tắt dần.

Khi đi thi

- Đọc định dạng đầu vào; ID là chỉ số, không chuẩn hóa.
- Lưu vocabulary, PAD/UNK, `max_len` cùng checkpoint.
- Baseline masked mean + linear trước RNN/Transformer.
- Thêm cột PAD: biểu diễn phải giữ nguyên.
- Đo phân phối độ dài trước khi chọn `max_len`.

05

Luyện đề

LUYỆN ĐỀ 1

Train word2vec (skip-gram) từ một bảng ngẫu nhiên, trên nhiều bài tường thuật bóng đá. Train xong, cosine giữa “thắng” và “thua” rất cao. Giải thích nào đúng?

A Train bị lỗi, vì skip-gram phải đẩy hai từ trái nghĩa ra xa nhau

B Hai từ có ID kề nhau, nên khởi tạo đã đặt chúng gần nhau

C Mục tiêu là đoán các từ xung quanh, mà hai từ này có ngữ cảnh rất giống nhau

D Skip-gram học theo nhãn chủ đề, và cả hai từ cùng nhãn thể thao

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 218

LUYỆN ĐỀ 2

Ba biến thể RNN, bỏ bias, với trạng thái $\mathbf{s}_t \in \mathbb{R}^m$ và đầu vào $\mathbf{x}_t \in \mathbb{R}^d$. (1) $\mathbf{s}_t = f(W_{ss}\mathbf{s}_{t-1} + W_{sx}\mathbf{x}_t)$. (2) $\mathbf{s}_t = f\left(W_{ssx}[\mathbf{s}_{t-1}; \mathbf{x}_t]\right)$, với W_{ssx} có shape $m \times (m+d)$. (3) $\mathbf{s}_t = f(W_{ss}\mathbf{s}_{t-1}) + f(W_{sx}\mathbf{x}_t)$. Với $f = \tanh$, câu nào đúng?

A Cả ba luôn tương đương, với mọi hàm f

B (1) và (2) tương đương, với $W_{ssx} = [W_{ss}\ W_{sx}]$

C (1) và (3) tương đương, vì tanh là hàm lẻ

D Không cặp nào tương đương, khi f là hàm phi tuyến

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q9(a)

LUYỆN ĐỀ 3

Một RNN thường phân loại các review dài khoảng 400 token. Nhãn thường do câu đầu quyết định, kiểu “Không nên mua: …”, nhưng model chỉ học được tín hiệu từ vài chục token cuối. Nguyên nhân chính là gì, và sửa theo hướng nào?

A Gradient về tới token đầu bị tắt dần qua nhiều bước, nên dùng LSTM, GRU hoặc attention

B Trạng thái ẩn quá nhỏ để chứa 400 token, nên tăng hidden size lên gấp bốn

C Dữ liệu lệch về các token gần cuối, nên cân bằng lại số mẫu của từng nhãn

D Learning rate quá nhỏ cho token đầu, nên tăng riêng learning rate của embedding

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q2.3

Chốt hợp đồng

ID lookup → padding/mask → pooling đúng số token.

Mean bỏ thứ tự; RNN giữ thứ tự nhưng đọc tại `lengths-1`.

Chuỗi dài: gradient tắt dần, LSTM có đường cộng.

Làm **10 câu kiểm tra cuối bài**; trong notebook, thêm cột PAD và kiểm biểu diễn không đổi.